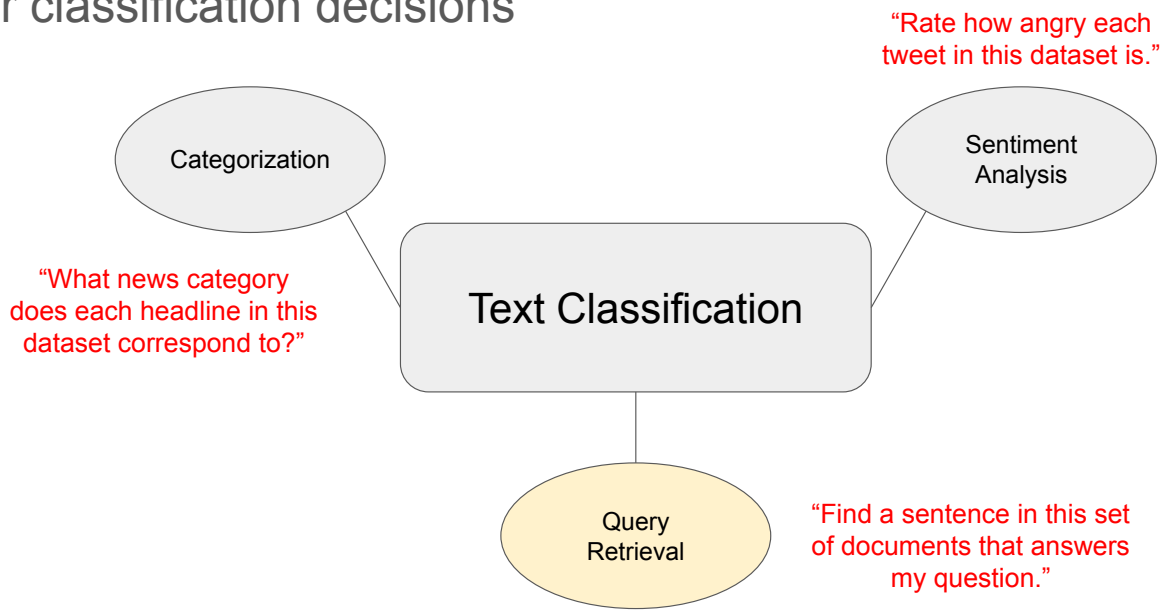
BERT-Augmented LLM Text Classification Tasks

Josh Greenbaum, Alex McFarland, Aiden DeBoer

Benefits of using LLMs for Text Classification

- Does not require any hand-labeled data
- Does not require any specific pre-training (just a good prompt)
- Can offer explanations for classification decisions
- Easy to implement
- High Accuracy



Downside of using LLMs for Text Classification

COST

- Extremely expensive at scale
- Recursive LLM frameworks amplify costs

Comparative Pricing for GPT and Claude 2			
LLM Model	Prompt (per 1,000 tokens)	Completion (per 1,000 tokens)	Cost per Document Processed
GPT-3.5 Turbo (4K)	$0.0015	$0.0020	$0.0034
Claude Instant (100K)	$0.0016	$0.0055	$0.0044
GPT-3.5 Turbo (16K)	$0.0030	$0.0040	$0.0068
Claude 2 (100K)	$0.0110	$0.0327	$0.0286
GPT 4 (8K)	$0.0300	$0.0600	$0.0720
GPT 4 (32K)	$0.0600	$0.1200	$0.1440

Recursive LLMs (from Zhang, Kraska, Khattab, 2026)

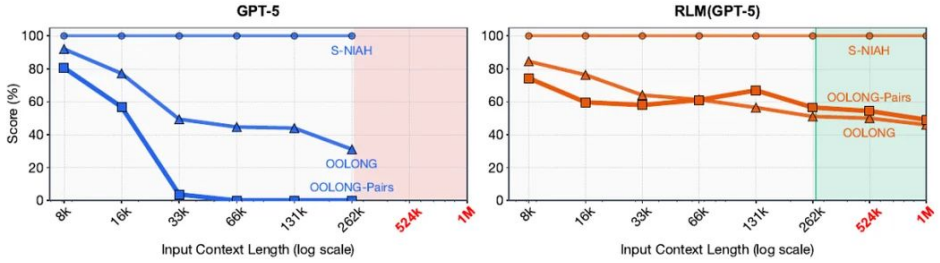
https://arxiv.org/pdf/2512.24601

Instead of processing an entire long input at once, a Recursive Language Model (RLM) treats the input as a structured object and recursively invokes itself (the RLM) on smaller, relevant subcomponents.

Rather than relying on a single massive context window, the model:

- Decomposes the problem into smaller pieces
- Selectively analyzes relevant sections
- Recursively calls itself on subparts

This approach works on 10M+ token inputs, far more than a traditional LLM context window.



Model	CodeQA	BrowseComp+ (1K)	OOLONG	OOLONG-Pairs
Task Length N (tokens)	23K-4.2M	6M-11M	131K	32K
Qwen3-Coder-480B				
Base Model	20.00* ($0.13 ± $0.08)	0.00* (N/A) ± (N/A)	36.00 ($0.06 ± $0.00)	0.06 ($0.05 ± $0.01)
CodeAct (+ BM25)	24.00* ($0.17 ± $0.08)	12.66 ($0.39 ± $0.50)	38.00 ($1.51 ± $1.09)	0.28 ($1.54 ± $0.35)
Summary agent	50.00 ($1.26 ± $1.50)	38.00 ($8.98 ± $2.12)	44.06 ($0.15 ± $0.01)	0.31 ($0.05 ± $0.00)
RLM	56.00 ($0.92 ± $1.23)	44.66 ($0.84 ± $0.63)	**48.00** ($0.61 ± $0.49)	**23.11** ($1.02 ± $0.52)
RLM (no sub-calls)	**66.00** ($0.18 ± $0.58)	**46.00** ($0.82 ± $0.69)	43.50 ($0.32 ± $0.13)	17.34 ($1.77 ± $1.23)
GPT-5				
Base Model	24.00* ($0.13 ± $0.07)	0.00* (N/A) ± (N/A)	44.00 ($0.14 ± $0.02)	0.04 ($0.16 ± $0.10)
CodeAct (+ BM25)	22.00* ($0.06 ± $0.08)	51.00 ($0.71 ± $1.20)	38.00 ($0.61 ± $1.06)	24.67 ($0.75 ± $0.43)
Summary agent	58.00 ($1.31 ± $1.46)	70.47 ($0.57 ± $0.10)	46.00 ($0.13 ± $0.01)	0.01 ($0.13 ± $0.09)
RLM	**62.00** ($0.11 ± $0.10)	**91.33** ($0.99 ± $1.22)	**56.50** ($0.43 ± $0.85)	**58.00** ($0.33 ± $0.20)
RLM (no sub-calls)	58.00 ($0.18 ± $0.56)	88.00 ($0.44 ± $0.90)	36.00 ($0.37 ± $0.42)	43.93 ($0.69 ± $1.16)

Our Version of a RLM

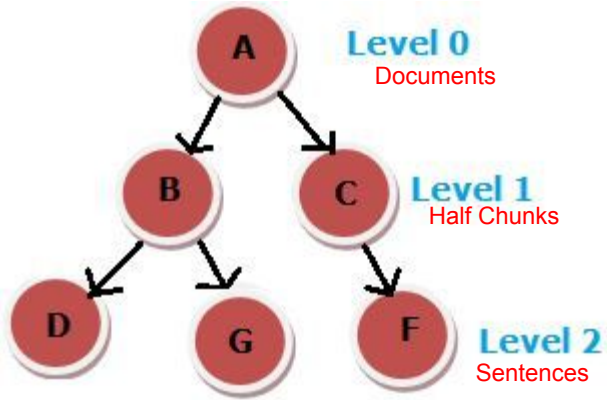
Since the version of the RLM we showed on the last slide already has a mechanism for deciding which branches to explore, we will do our own simple recursion ourselves.

We model document classification as a hierarchical search problem:

- Nodes represent text chunks.
- Edges represent recursive decomposition.
- Leaves represent minimal evaluation units (sentences/paragraphs).

At each node:

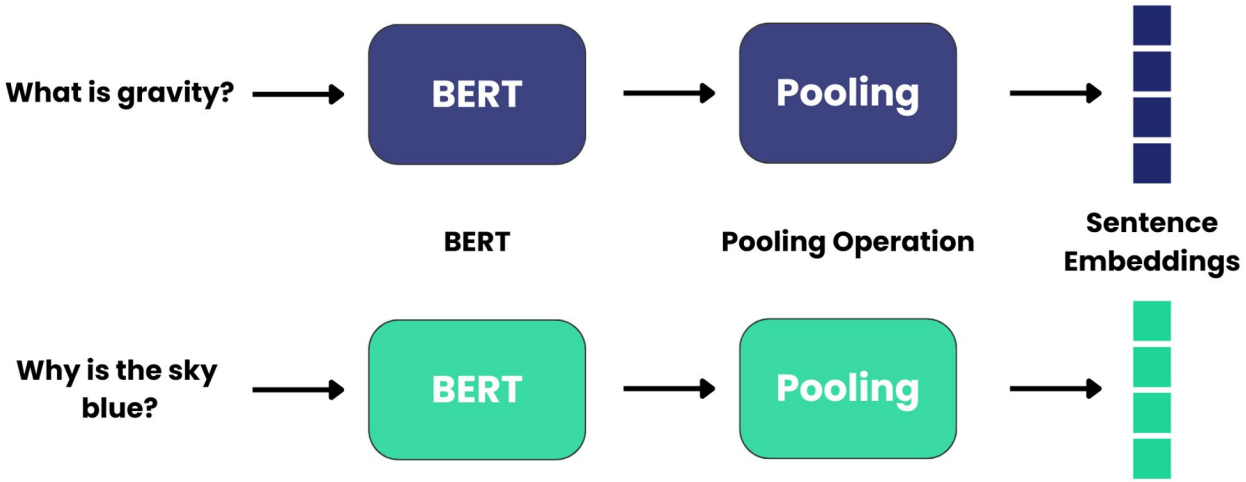
- Apply embedding-based gating (cheap).
- Only invoke LLM classification when necessary (expensive).



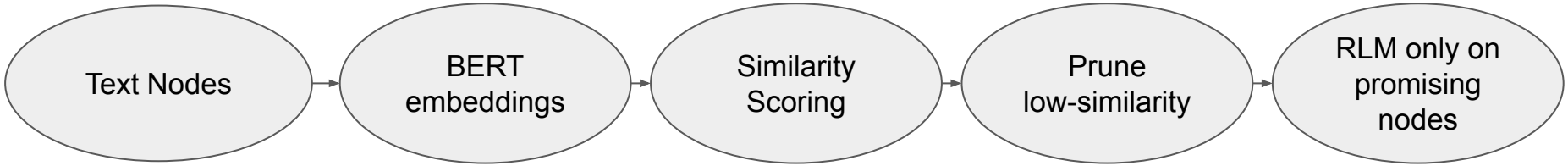
BERT

https://www.geeksforgeeks.org/nlp/explanation-of-bert-model-nlp/

BERT converts text into contextual vector representations. After pooling, we obtain a single sentence embedding that captures the semantic meaning of the input, enabling efficient similarity comparison between texts.



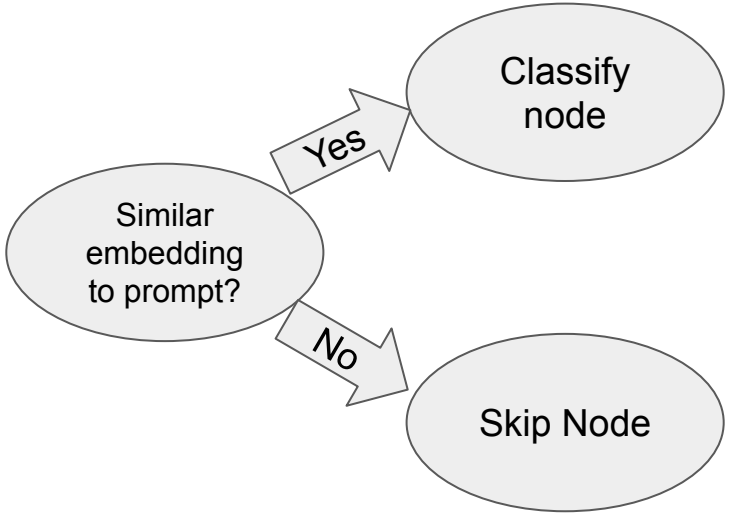
Our Idea: Use BERT as a Recursion Gating Mechanism



- Embeddings are cheap (anywhere from 5x-20x cheaper than LLM calls on average)
- RLM calls are expensive
- Use cheap signal to avoid expensive calls

Intuition and Assumptions

- Similar texts lead to similar embeddings
- Different embeddings likely lead to different LLM classifications
- Therefore, we can predictively skip many nodes, leading to less recursive calls.



Proposed Algorithm

1

Compute BERT Embeddings

Compute BERT embeddings for each candidate document.

2

Compute BERT similarity

Compute BERT similarity between each candidate document and the query prompt.

3

Prune Documents

Remove documents with low similarity from candidate pool

4

Divide Remaining Documents

Split Remaining documents into chunks, and compute BERT for each chunk.

5

Prune Again, Recur

Prune chunks that have low estimated scores. Repeat 4 & 5 until chunks contain only one sentence/paragraph.

6

Call LLM

Call the LLM classification on remaining chunks and label document based on chunk result.

Evaluation Plan (Measuring Efficiency vs. Accuracy)

We will compare the classification performance on **run time**, **LLM call count**, and **classification accuracy** of three different configurations:

1. Full Recursive LLM (No Pruning)
2. BERT Guided Pruning (Our Method)
3. Random Pruning (Control)

Risks and Open Questions

- Does embedding similarity truly predict LLM output?
- How sensitive is pruning threshold?
- What kind of pruning is best?
- Does recursion amplify error, and by how much?
- How does our recursion compare to a full RLM?